

Aula 2 - Modelagem Software Orientado a Objetos

Esta apresentação explora a modelagem de sistemas. Faremos isso a partir de código e requisitos. Aprenda a criar diagramas de caso de uso. Descubra como modelar sistemas de forma eficaz.

G por Giovanni Amorim





Modelagem a partir de Código Existente



Análise de Código

Examine o código.
Identifique atores e casos de uso.



Mapeamento

Mapeie as relações entre os elementos.

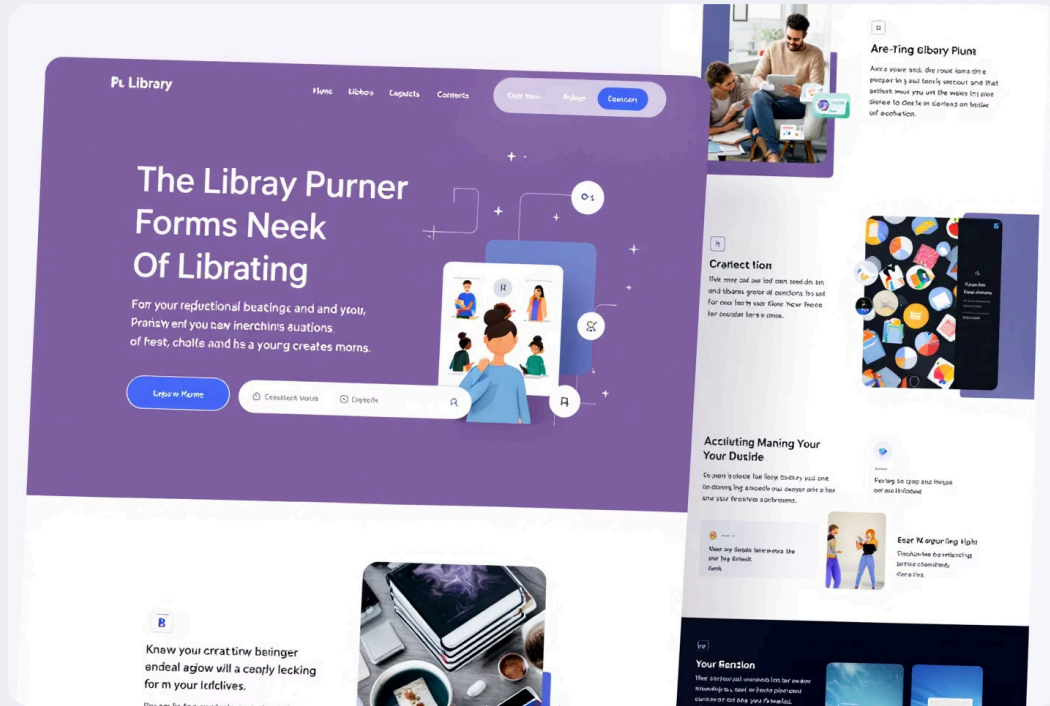


Diagramação

Use o Visual Paradigm para criar o diagrama.

Comece com um trecho de código simples. Um sistema de login é um bom exemplo. Modele o sistema a partir desse código.

Modelagem a partir de Requisitos



Diagramação

Crie o diagrama de caso de uso no Visual Paradigm. Detalhe as interações.

Análise de Requisitos

Avalie os requisitos do sistema. Identifique os atores e seus papéis.

Use os requisitos para criar um diagrama. Um sistema de biblioteca online é um bom exemplo. Modele as funções de busca, cadastro e reserva.

Modelagem a partir de Tabelas de Banco de Dados

Vamos entender este processo através de um exemplo prático de um sistema de e-commerce:

Análise das Tabelas

Exemplo: Tabelas 'produtos', 'clientes' e 'pedidos'. A tabela 'pedidos' tem chave estrangeira referenciando 'clientes' (cliente_id) e uma tabela intermediária 'itens_pedido' relacionando com 'produtos'.

1

Identificação de Entidades

Transformamos em classes: Produto (id, nome, preco, estoque), Cliente (id, nome, email, endereco) e Pedido (id, data, status, cliente_id).

2

Mapeamento de Atributos

Na classe Produto: private Long id; private String nome; private BigDecimal preco; private Integer estoque; Com getters/setters apropriados.

3

Definição de Relacionamentos

Cliente -> Pedido (1:N): Um cliente pode ter vários pedidos. Pedido -> Produto (N:N): Um pedido pode ter vários produtos e um produto pode estar em vários pedidos.

4

Dica: Neste exemplo, a tabela 'itens_pedido' não se torna uma classe principal, mas sim um relacionamento entre Pedido e Produto.

Modelagem a partir de Código (Diagrama de Caso de Uso)

A partir do código abaixo desenvolva o diagrama de caso de uso. Utilize a Ferramenta VisualParadigma, depois faça em seu caderno.



```
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

class SistemaLogin {
    private Map<String, String> usuarios;

    public SistemaLogin() {
        usuarios = new HashMap<>();
        // Adicionando um usuário administrador para teste
        usuarios.put("admin", "1234");
    }

    public boolean fazerLogin(String usuario, String senha) {
        if (usuarios.containsKey(usuario) && usuarios.get(usuario).equals(senha)) {
            System.out.println("Login realizado com sucesso!");
            return true;
        } else {
            exibirMensagemErro(); // <<extend>> - ocorre apenas se o login falhar
            return false;
        }
    }

    private void exibirMensagemErro() {
        System.out.println("Erro: Usuário ou senha inválidos!");
    }

    public void adicionarUsuario(String admin, String novoUsuario, String novaSenha) {
        if (fazerLogin(admin, usuarios.get(admin))) { // <<include>> - login necessário
            usuarios.put(novoUsuario, novaSenha);
            System.out.println("Usuário " + novoUsuario + " adicionado com sucesso!");
        }
    }

    public void removerUsuario(String admin, String usuarioRemover) {
        if (fazerLogin(admin, usuarios.get(admin))) { // <<include>> - login necessário
            if (usuarios.containsKey(usuarioRemover)) {
                usuarios.remove(usuarioRemover);
                System.out.println("Usuário " + usuarioRemover + " removido com sucesso!");
            } else {
                System.out.println("Usuário não encontrado.");
            }
        }
    }
}

public class Main {
    public static void main(String[] args) {
        SistemaLogin sistema = new SistemaLogin();
        Scanner scanner = new Scanner(System.in);

        System.out.print("Digite o usuário: ");
        String usuario = scanner.nextLine();

        System.out.print("Digite a senha: ");
        String senha = scanner.nextLine();

        if (sistema.fazerLogin(usuario, senha)) {
            System.out.println("Acesso permitido!");

            // Testando funcionalidades administrativas
            sistema.adicionarUsuario("admin", "novoUsuario", "senha123");
            sistema.removerUsuario("admin", "novoUsuario");
        } else {
            System.out.println("Acesso negado.");
        }

        scanner.close();
    }
}
```

Criação de Diagrama de Caso de Uso

Análise do Código

Estudo detalhado do código fonte para compreender sua estrutura e funcionamento

Identificação dos Elementos

Identificação dos atores (X, Y) e casos de uso (A, B, rC)

Modelagem no Visual Paradigm

Criação do diagrama de caso de uso utilizando a ferramenta Visual Paradigm

Discussão Comparativa

Comparação entre os diferentes diagramas criados pelos alunos para enriquecer o aprendizado

Esta atividade permite compreender como transformar código em uma representação visual de suas funcionalidades através da UML.

Sistema de Atendimento por Chatbot

1

Interação Inicial

Cliente interage com o Chatbot para esclarecer dúvidas frequentes

2

Análise de Resposta

Chatbot processa a pergunta e verifica se pode responder adequadamente

3

Encaminhamento

Se necessário, direciona para atendente humano quando não consegue resolver

4

Registro de Interações

Sistema registra todas as conversas para análise posterior

O sistema integra atendimento automatizado com suporte humano, garantindo que todas as interações sejam registradas para melhoria contínua do serviço.

Atividade: Diagrama de Caso de Uso – Sistema de Chatbot

Objetivo

Nesta atividade, você irá modelar um sistema de chatbot usando a UML, aplicando técnicas de requisitos funcionais, com foco em:

- Identificação de atores principais e secundários
- Desenvolvimento de casos de uso
- Aplicação de relações de extend

Tarefa 1: Identificação de Elementos

1 Identifique os atores primários e secundários do sistema de chatbot

2 Defina os casos de uso principais do sistema

Dica: Considere as diferentes interações entre usuários, chatbot e atendentes humanos.

Tarefa 2: Modelagem de Relacionamentos

Desenvolva os relacionamentos entre os casos de uso identificados:

- ✓ Estabeleça as relações do tipo `<<include>>` para funções obrigatórias
- ✓ Defina as relações do tipo `<<extend>>` para funcionalidades condicionais

Dica: Foque especialmente nas situações onde o atendimento precisa ser transferido do chatbot para um atendente humano.